

NDC { TechTown }



Introduction to Qt ROS

Architecture and Demo

shawn.rutledge @ qt.io



<https://github.com/TheQtCompanyRnD/qtros>



<https://github.com/ec1oud/qt-presentations/tree/robotics>



Introduction to Qt ROS - Architecture and Demo

Agenda

- Qt, Qt Quick, declarative programming
- What is ROS?
- ROS API stack
- ROS communication patterns
- Explanation of demo
- ROS communication patterns in the demo code
- Collision detection physics
- Generating a Digital Twin from URDF



<https://github.com/ec1oud/qt-presentations/tree/robotics>

What is Qt?

- in the beginning: a general-purpose C++ framework and widget toolkit
 - containers, network, ...
 - improvement on Xt, Athena, Motif etc.
- cross-platform (all the platforms)
- single-rooted C++ inheritance (QObject base)
- properties with accessors and notifier signals; connect()
- more recently: exploit the GPU, move beyond C++
- QML, Qt Quick, 3D, Physics, ...



```
import QtQuick
import QtQuick.Controls
import QtQuick.Controls.Universal
```

```
SplitView {
    orientation: Qt.Vertical
    width: 480; height: 640

    TextEdit {
        id: textEdit
        textFormat: Text.MarkdownText
        font { family: fontFamily
                pixelSize: fontSize * 0.7 } // from Slides.qml
        text: "
```

```
# Declarative Programming
- bind a property to an expression
- automatic updates
- syntax: CSS-like
- non-declarative fallback: JS signal handling
- interpreted or compiled
"
```

```
    SplitView.minimumHeight: 50
}
```

```
Item {
    SplitView.fillWidth: true
    SplitView.fillHeight: true
```

```
    Rectangle {
        anchors { fill: parent; margins: 50 }
        color: tapHandler.pressed ? "Green" : "LightGreen"
```

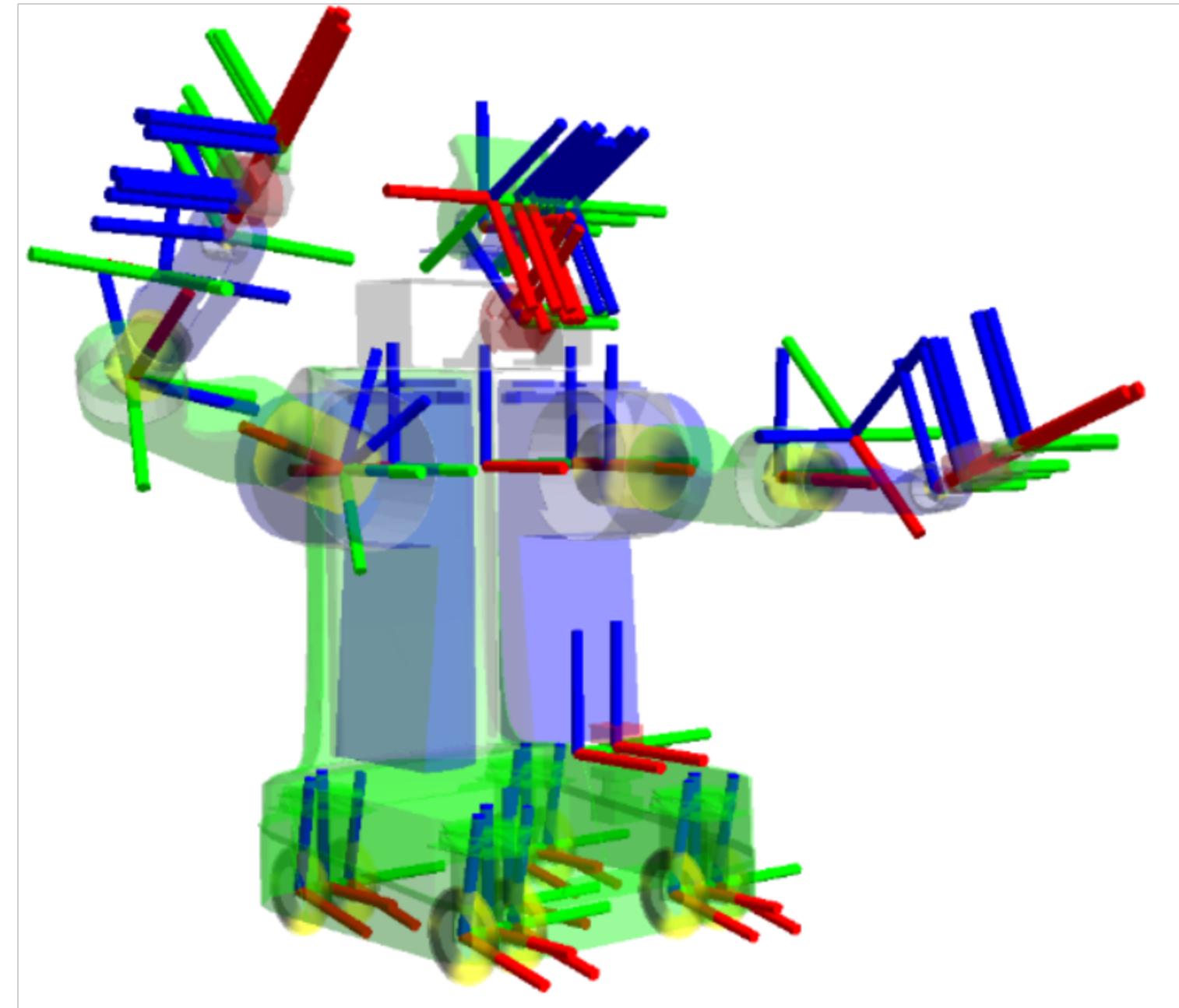
Declarative Programming

- bind a property to an expression
- automatic updates
- syntax: CSS-like
- non-declarative fallback: JS signal handling
- interpreted or compiled

174 characters

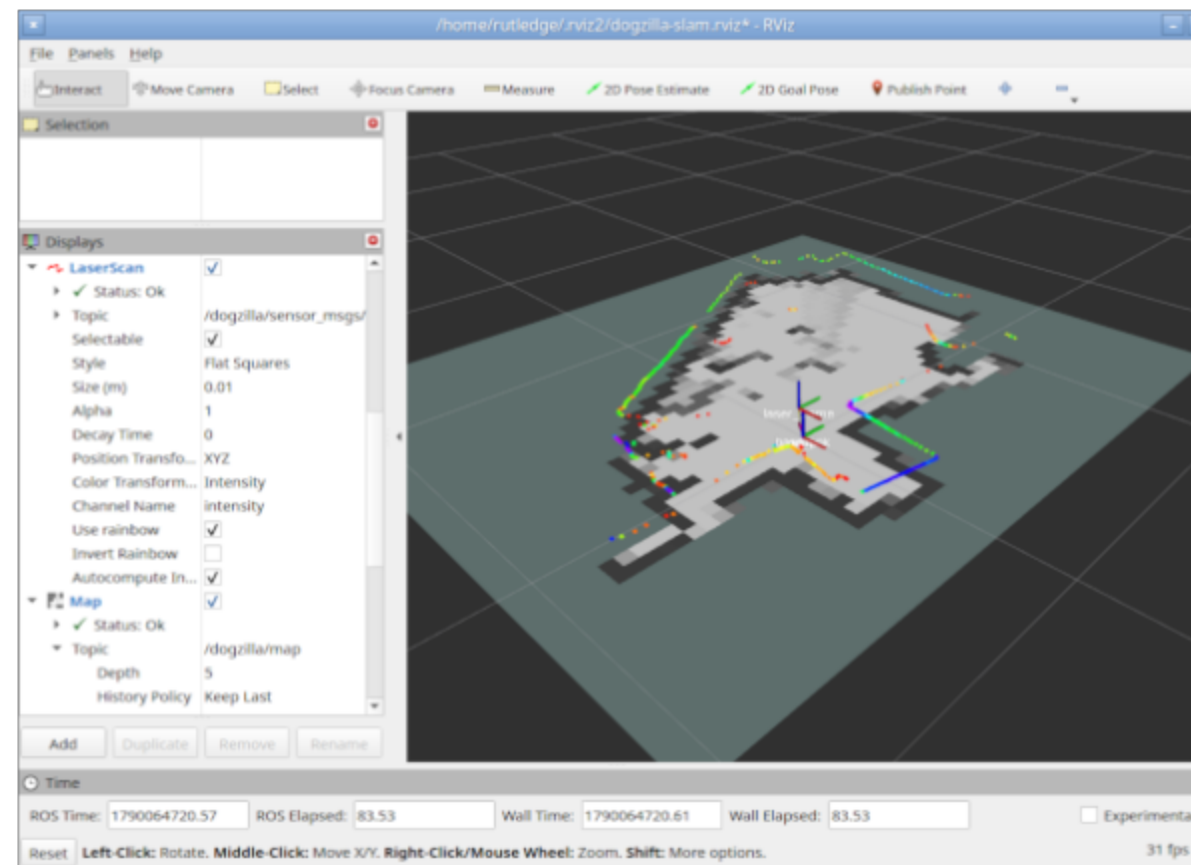
ROS: Robot "Operating System"

- communication middleware
 - ROS2: built on top of DDS (Data Distribution Service)
 - or Zenoh
 - discovery
- motion planning / kinematics
 - converting between frames of reference
 - collision checking
- navigation
 - path planning
 - obstacle avoidance
 - SLAM (Simultaneous Localization and Mapping)
- Ubuntu, Yocto, Windows, ...



Standard ROS utilities

- developer tools & visualization
 - RViz: 3D visualizer for robot, environment, LiDAR, planned paths
 - Gazebo: 3D simulator with physics
 - Foxglove Studio / RQt: older UI frameworks
- data logging: rosbag2
 - record and play back data
 - debug real-world sensor logs offline
- Python build tools
 - colcon: script to orchestrate building multiple packages
 - ament: enhancement for CMake/Python that defines how ROS packages are structured and discovered
 - EmPy: IDL to custom C++

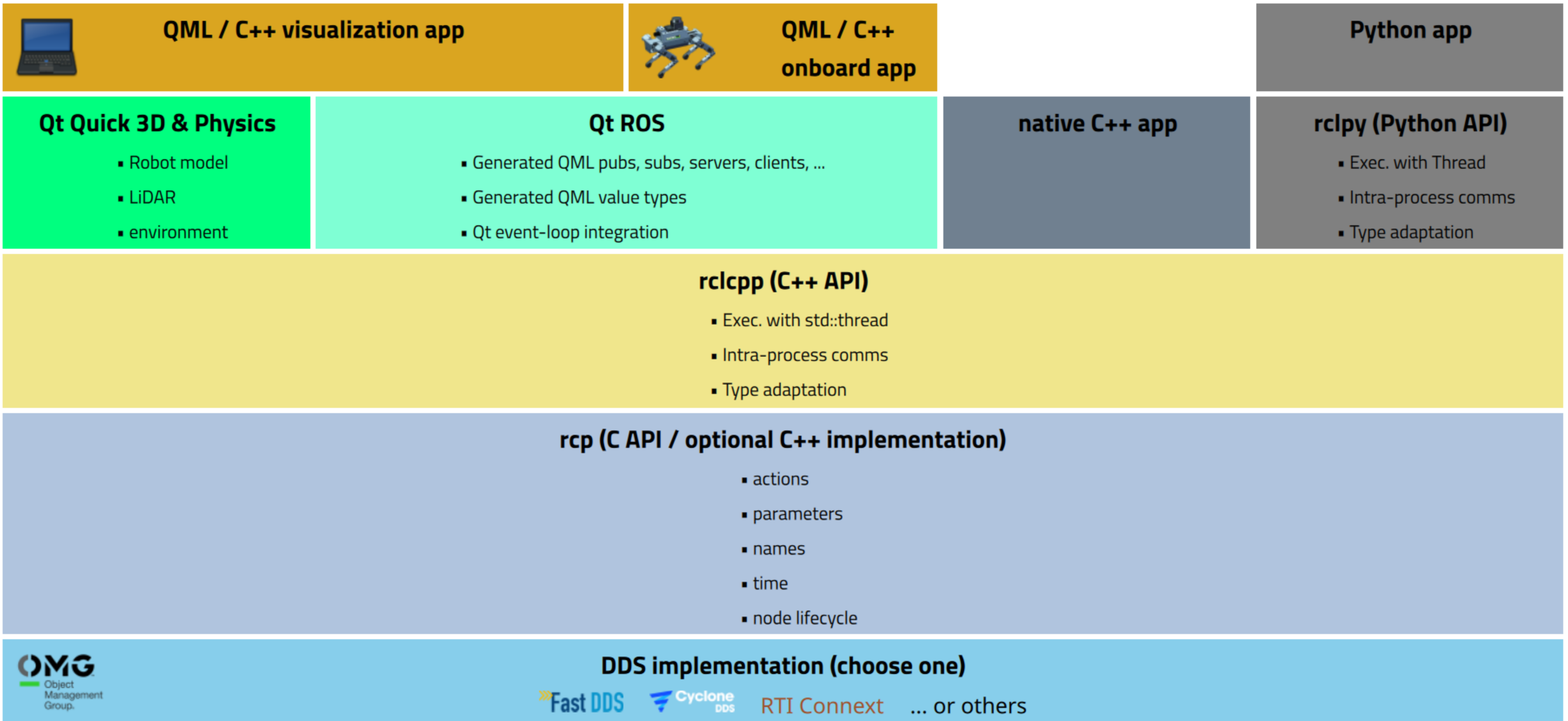


ROS versions

- LTS
 - Lyrical Luth (2026)
 - Jazzy Jalisco (2024)
- Rolling (untested)
- QtROS is generated



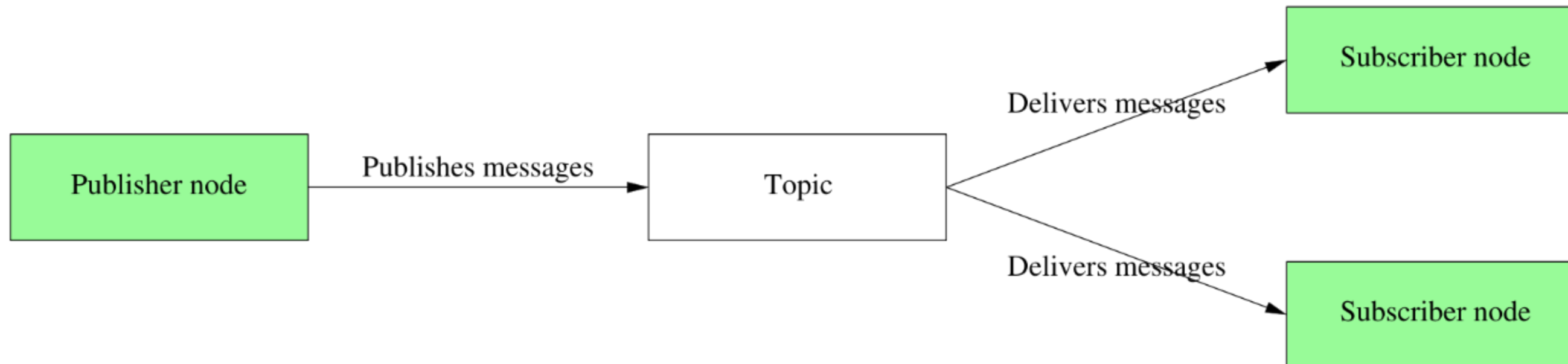
ROS 2 client API stack



ROS Communication Patterns

Publish/Subscribe on a Topic

- for continuous data streams: sensor data, status, ...
- topic definitions originate from .msg files
- asynchronous, one-way communication
- multiple publishers and subscribers



```
$ ros2 topic list
$ ros2 topic echo /dogzilla/joint_states
$ ros2 topic bw /dogzilla/joint_states
...
```

```

import QtQuick
import QtQuick.Controls
import QtQuick.Layouts
import QtRos2.GeometryMsgs

ColumnLayout {
    width: 200; height: 560

    Node { // in QtRos2.Core, not QQ3D
        id: rosNoded
        nodeName: "simpleSubscriber"

        PoseStampedSubscriber {
            id: sub
            property int receivedCount: 0
            topic: "/dogzilla/body_pose/state"
            onMessageReceived: ++receivedCount
        }
    }

    Label { text: `connected: ${sub.connected} received: ${sub.receivedCount}` }

    Label { text: `**roll** ${sub.pose.orientation.rpyDegrees.x.toFixed(2)}° ` +
        `**pitch** ${sub.pose.orientation.rpyDegrees.y.toFixed(2)}° ` +
        `**yaw** ${sub.pose.orientation.rpyDegrees.z.toFixed(2)}°`
        textFormat: Label.MarkdownText }

    TextArea {
        text: JSON.stringify(sub.pose, null, 2)
        Layout.columnSpan: 2; Layout.fillWidth: true; Layout.fillHeight: true
        readOnly: true
    }
}

```

connected: false received: 0

roll 0.00° **pitch** 0.00° **yaw** 0.00°

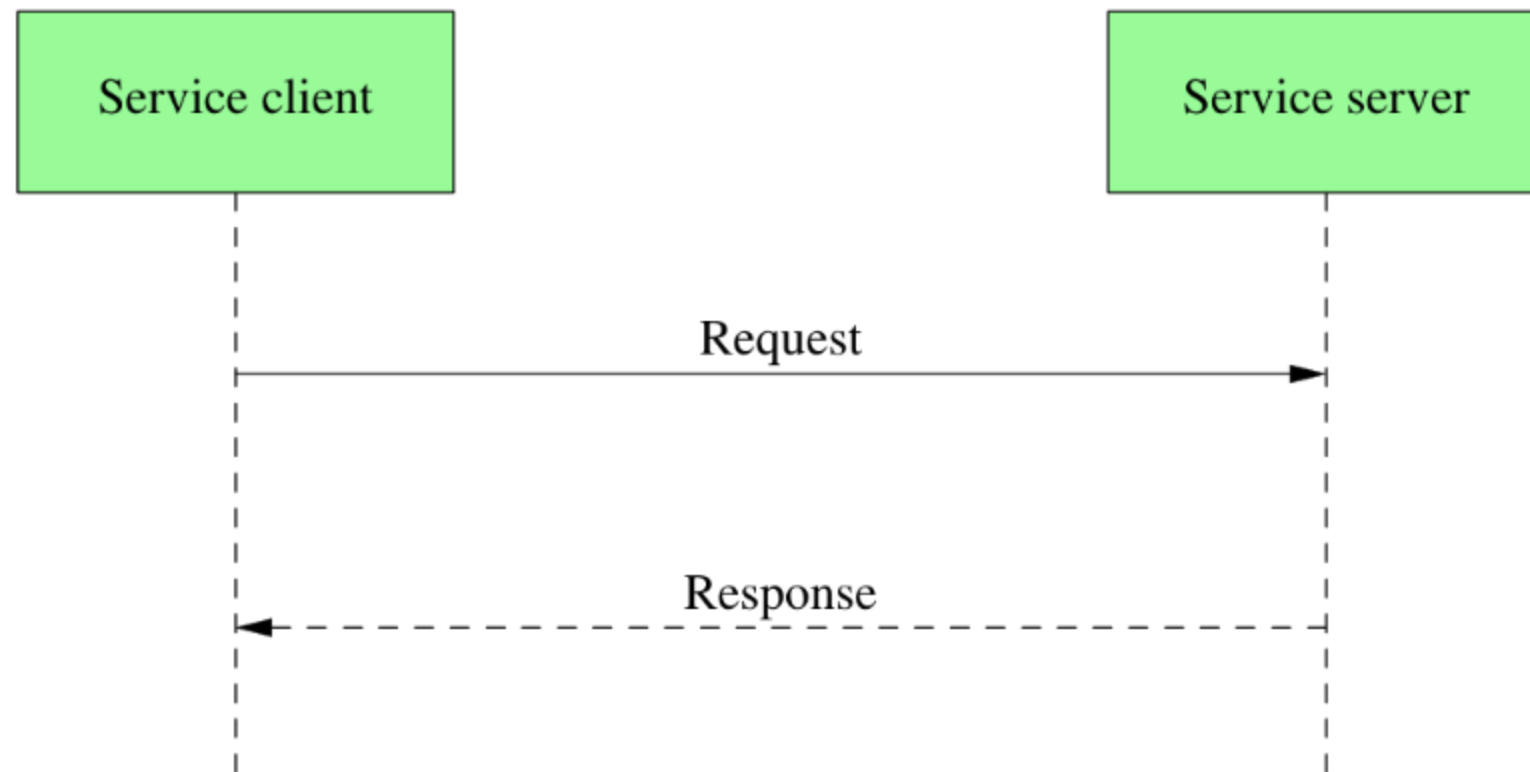
```

{
  "position": {
    "x": 0,
    "y": 0,
    "z": 0
  },
  "orientation": {
    "x": 0,
    "y": 0,
    "z": 0,
    "w": 1,
    "eulerAngles": {
      "x": 0,
      "y": 0,
      "z": 0
    }
  },
  "rpy": {
    "x": 0,
    "y": 0,
    "z": 0
  },
  "rpyDegrees": {
    "x": 0,
    "y": 0,
    "z": 0
  },
  "scalar": 1
}

```

Service

- synchronous request/response: e.g. a query
- service definitions originate from .srv files
- one-to-one
- return response

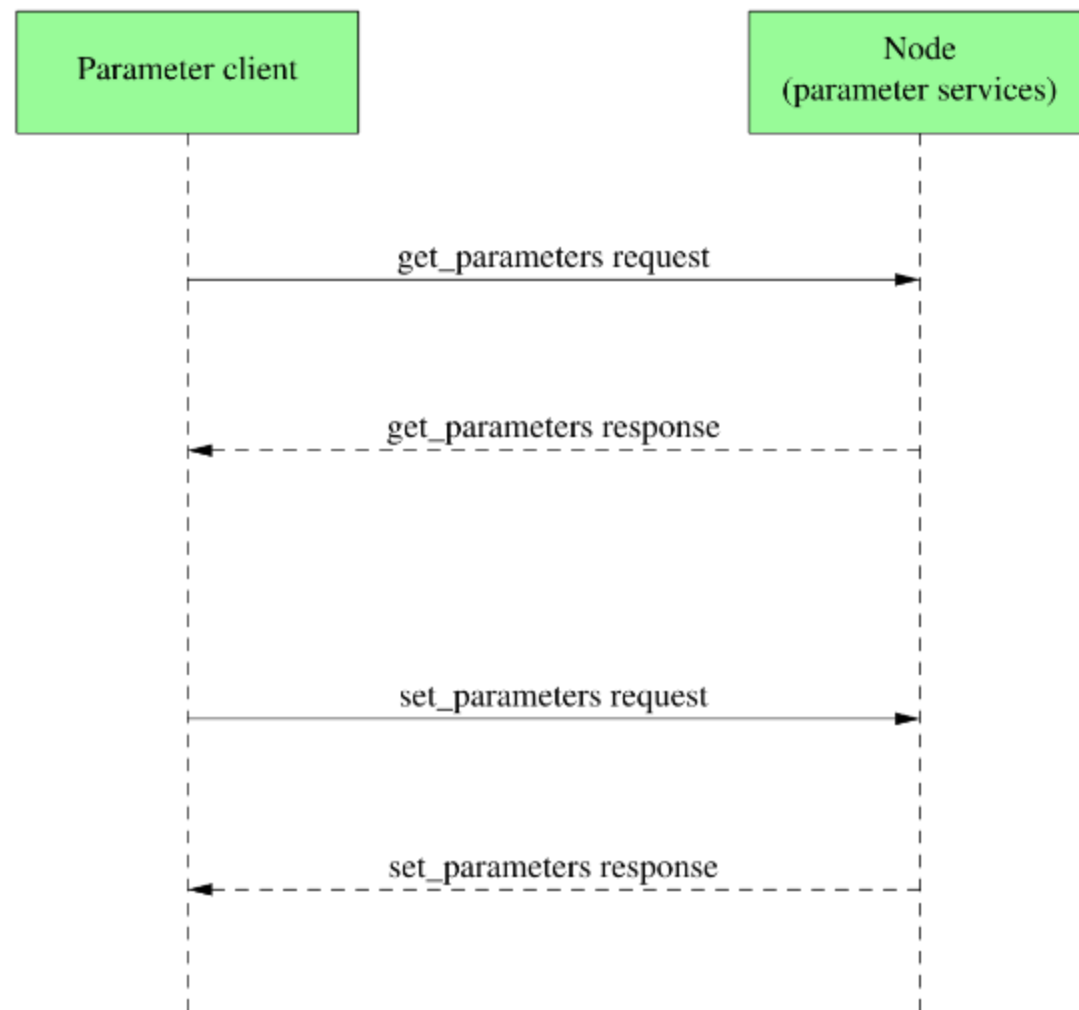


```
$ ros2 service list
```

```
$ ros2 service call /dogzilla/speech/stop std_srvs/srv/Trigger1
```

Parameter

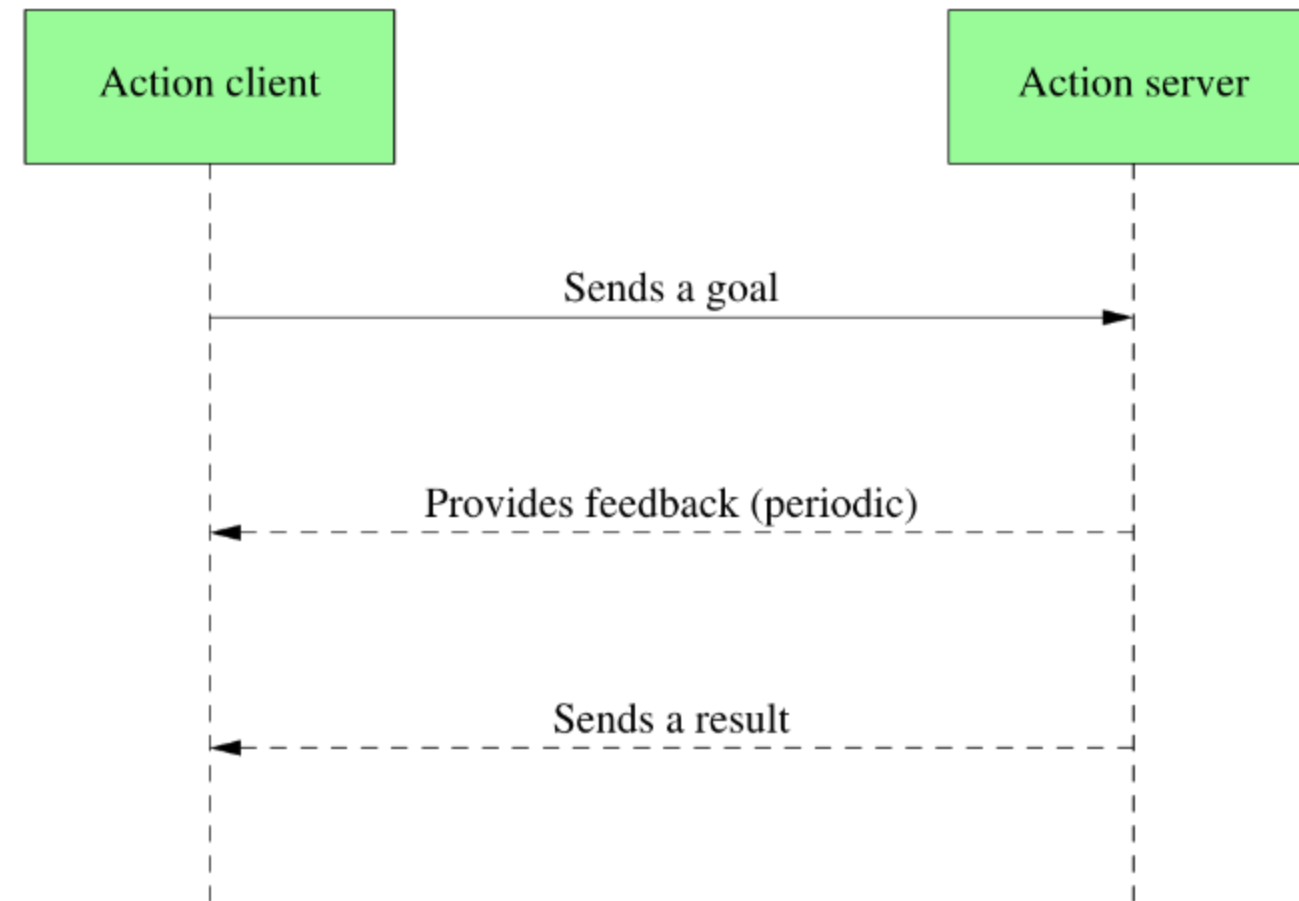
- configuration values stored per node, settable at startup or at runtime
- get/set through built-in services on the node: e.g. `get_parameters`, `set_parameters`
- synchronous request/response, same protocol as a regular service
- typed: `bool`, `int64`, `float64`, `string`, `byte[]` and array variants



```
$ ros2 param list
$ ros2 param describe /dogzilla/dogzillad motors.engaged
$ ros2 param get /dogzilla/dogzillad audio.mic
$ ros2 param set /dogzilla/dogzillad audio.master 0.8
```

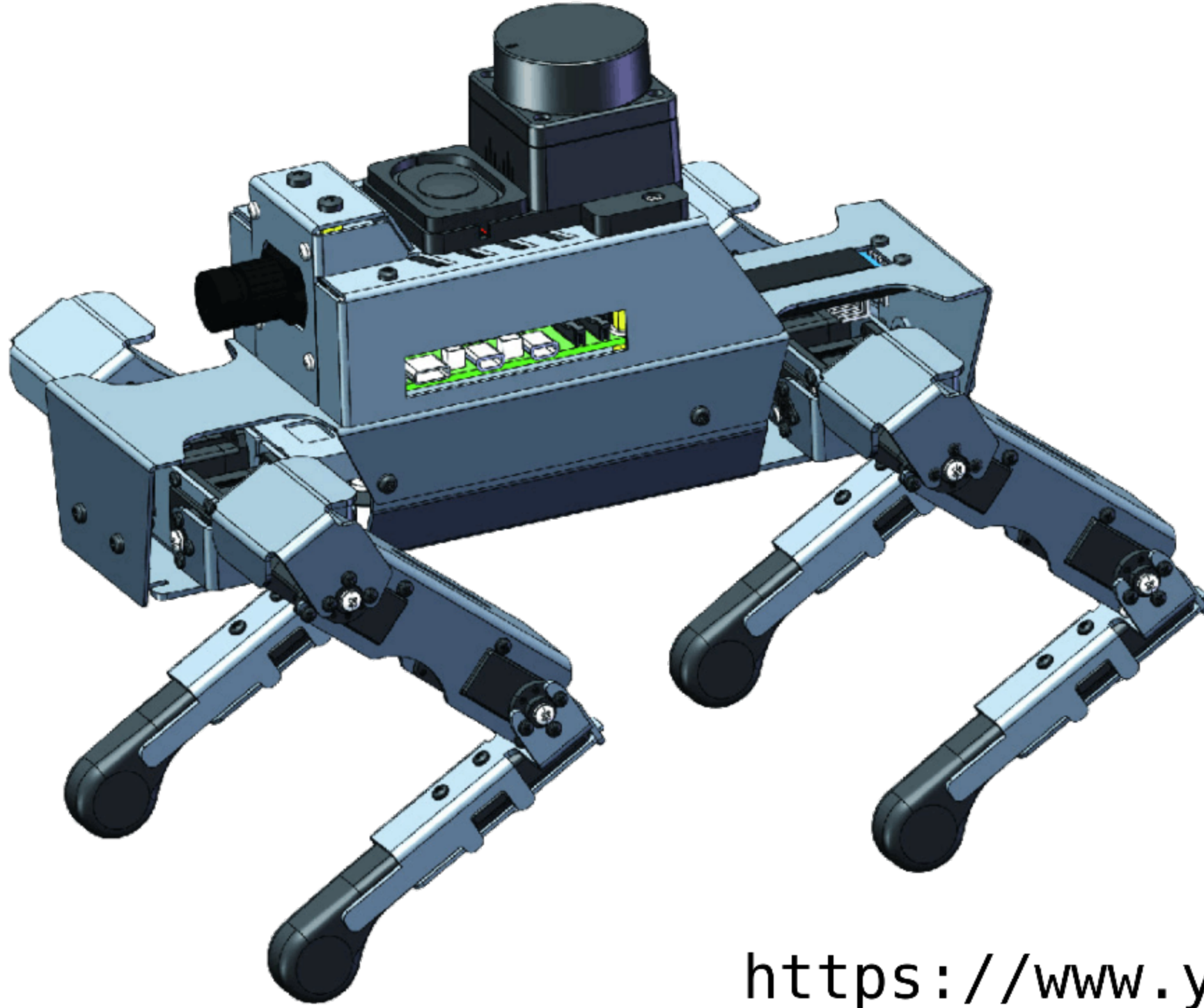
Action

- goal with feedback and result: e.g. robot motion
- action definitions originate from .action files
- asynchronous, one-to-one
- cancel if needed
- return result on success



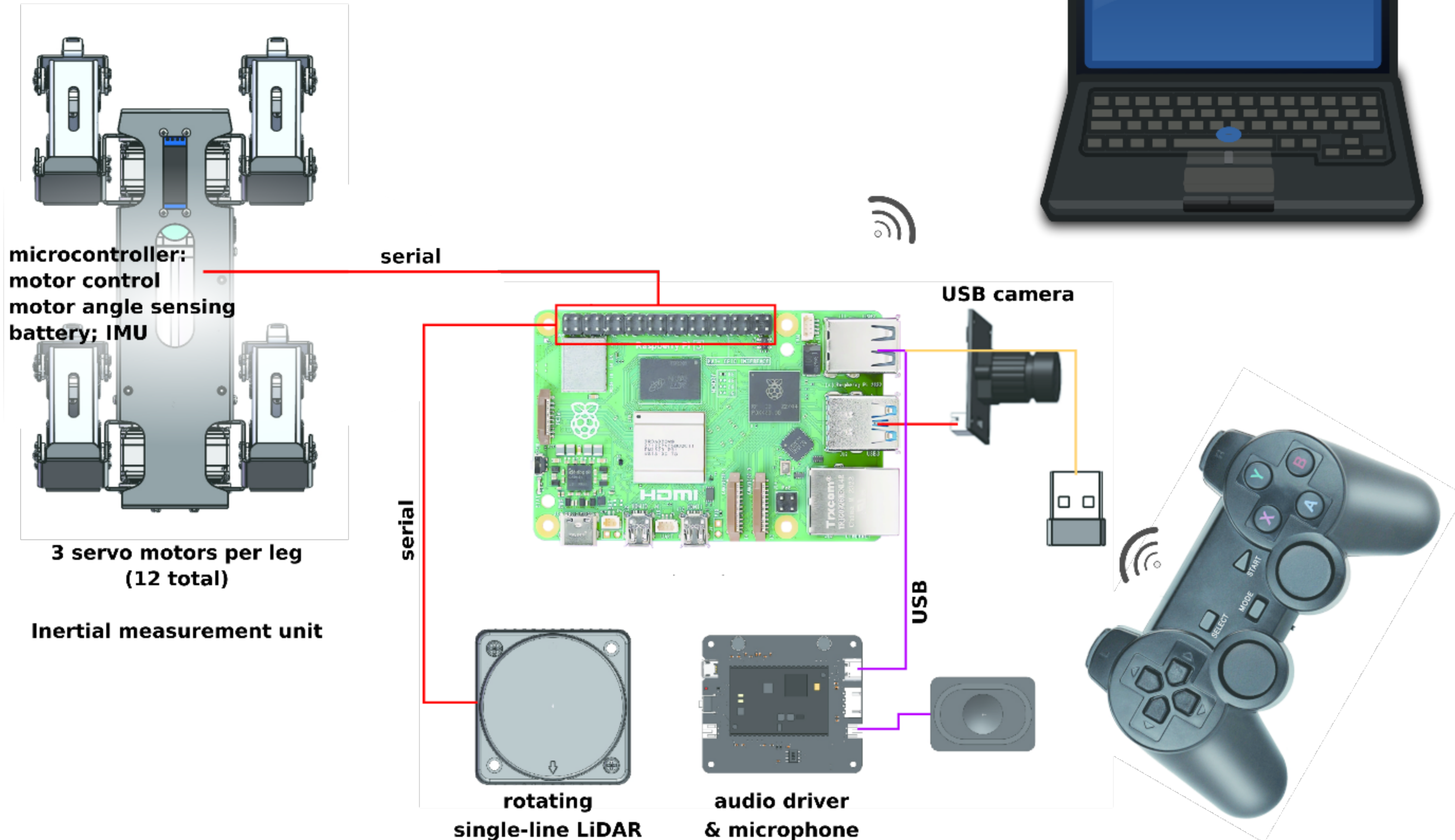
`$ ros2 action list`

Demo Robot: Dogzilla S2

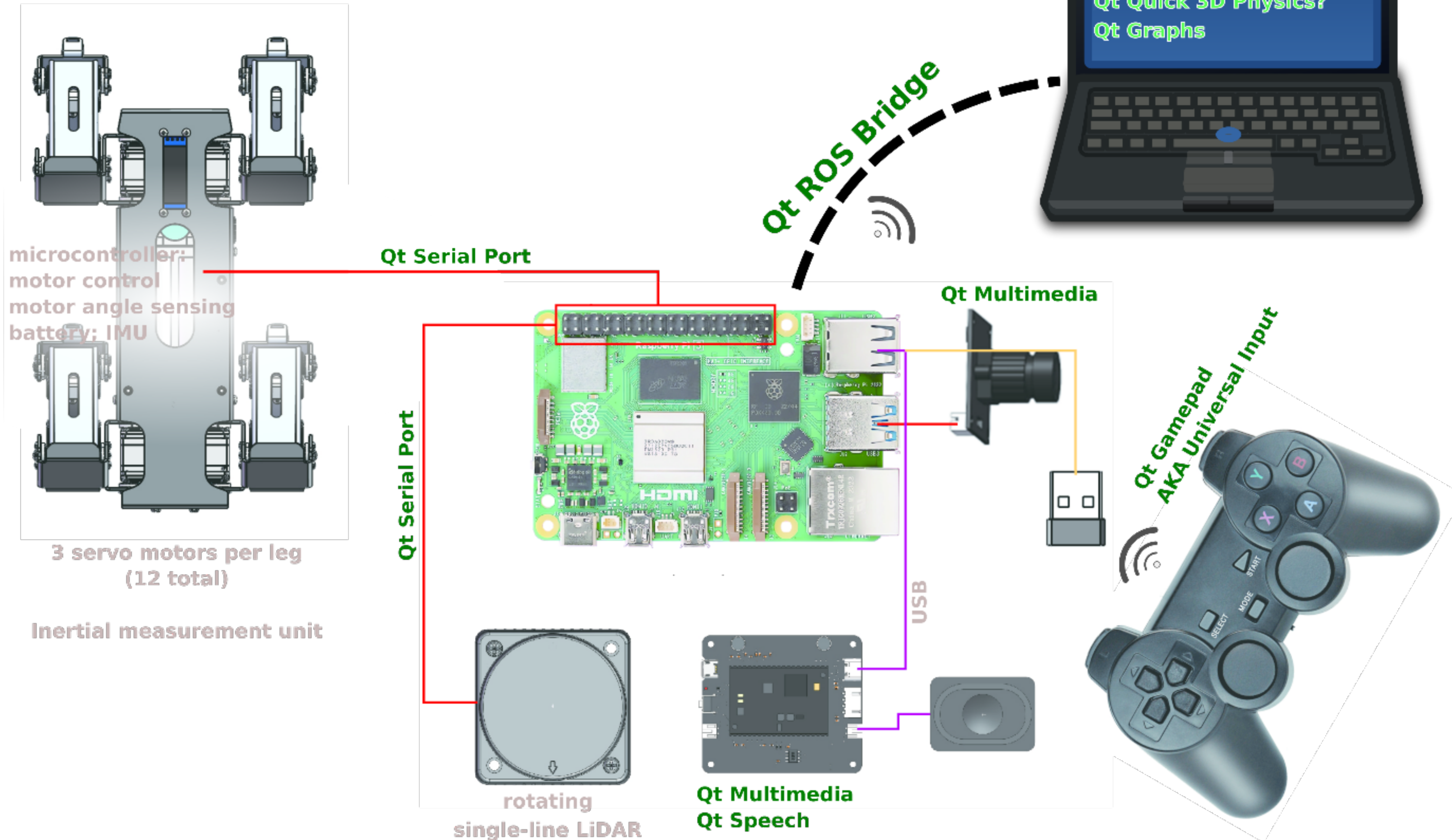


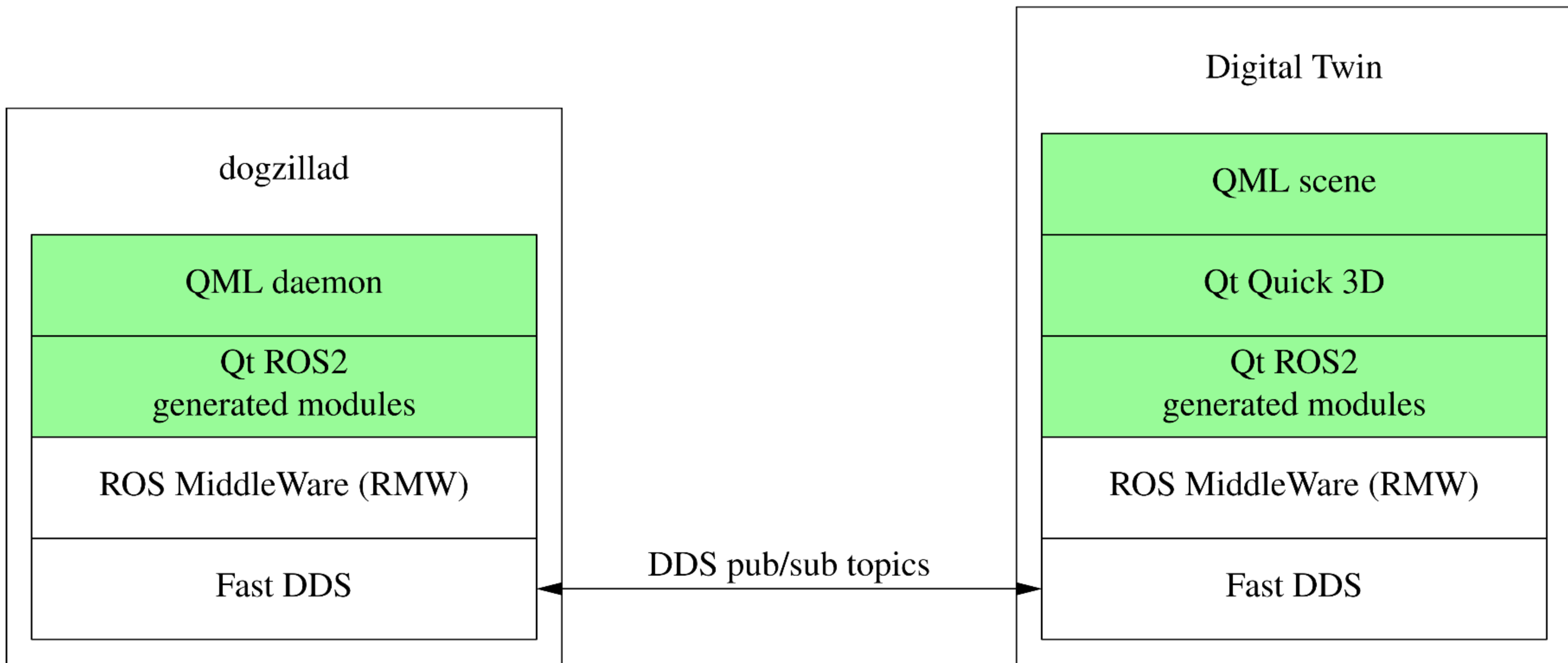
<https://www.yahboom.net/study/D0GZILLA>

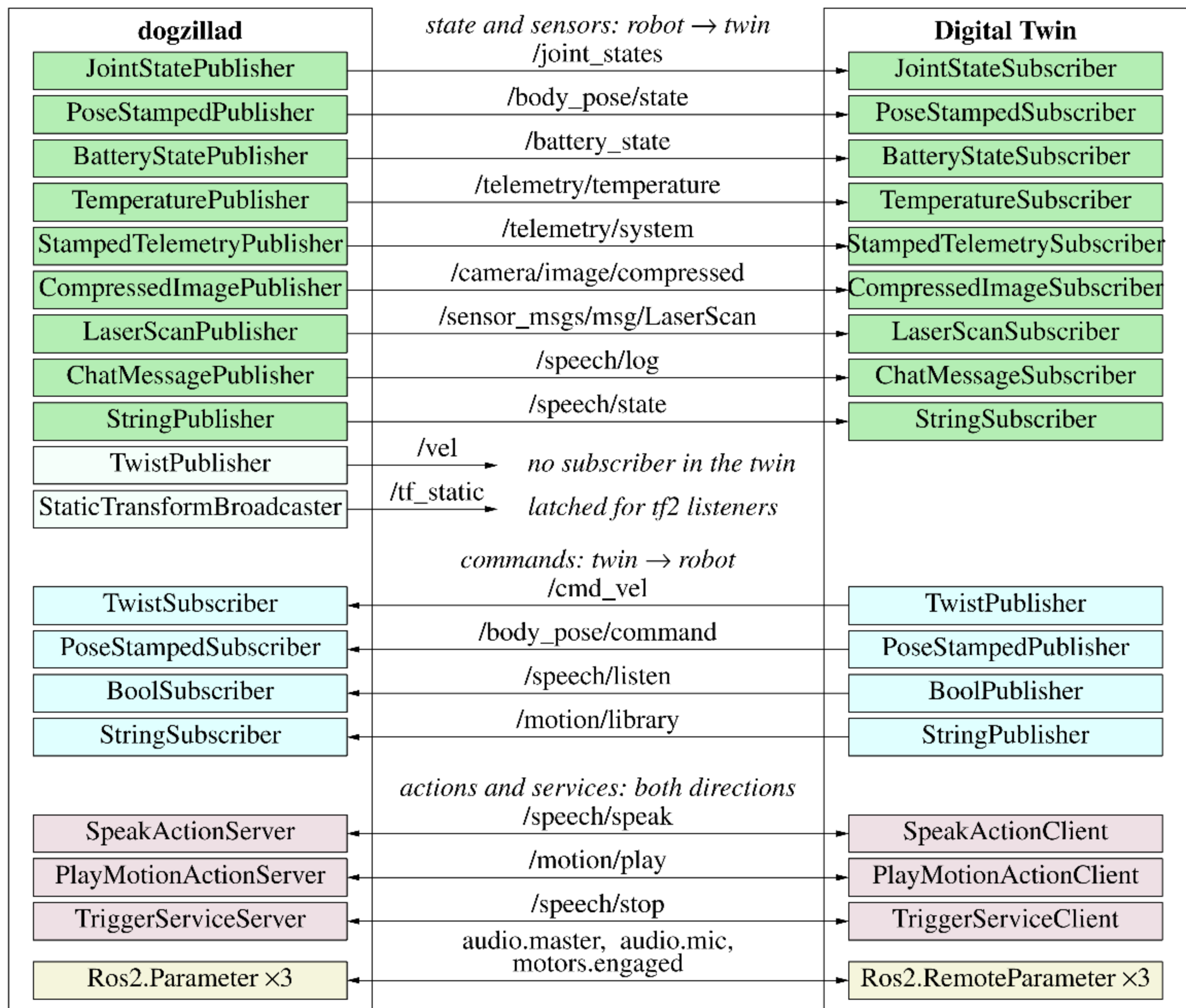
Demo hardware components



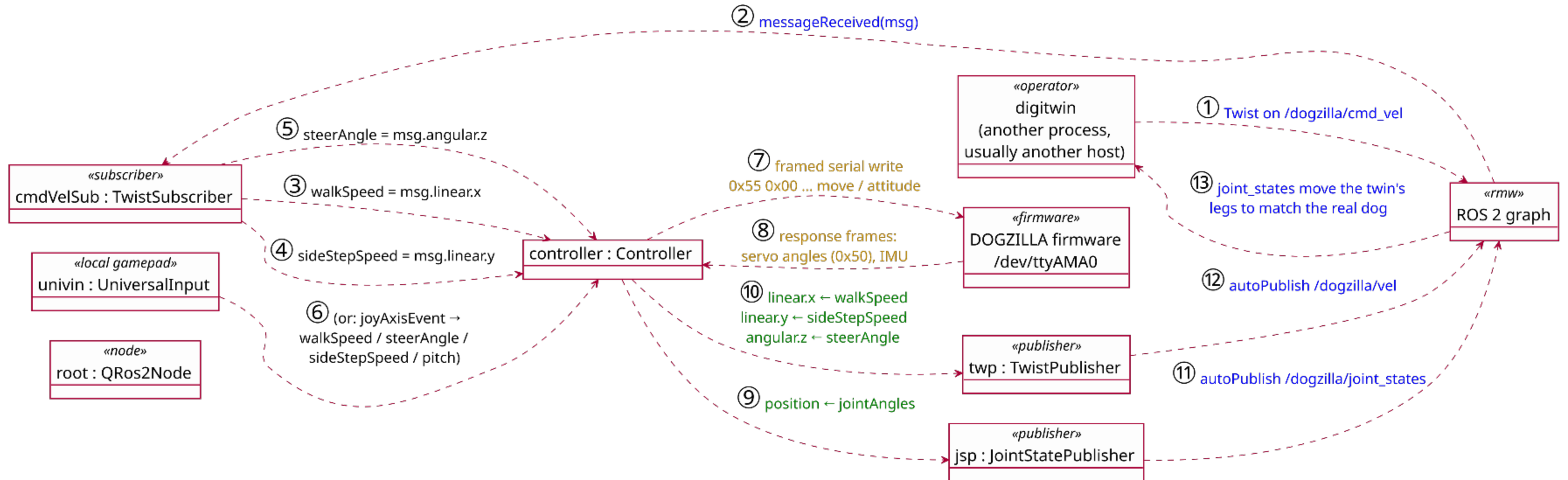
How Qt modules are used







Motion control in dogzillad



```

import QtQml // not QtQuick
import QtMultimedia
import QtTextToSpeech
import QtUniversalInput
import Dogzilla
import Dogzilla.Interfaces
import QtRos2.Core as Ros2
import QtRos2.GeometryMsgs
import QtRos2.SensorMsgs
import QtRos2.StdMsgs
import QtRos2.StdSrvs
import QtRos2.Transforms

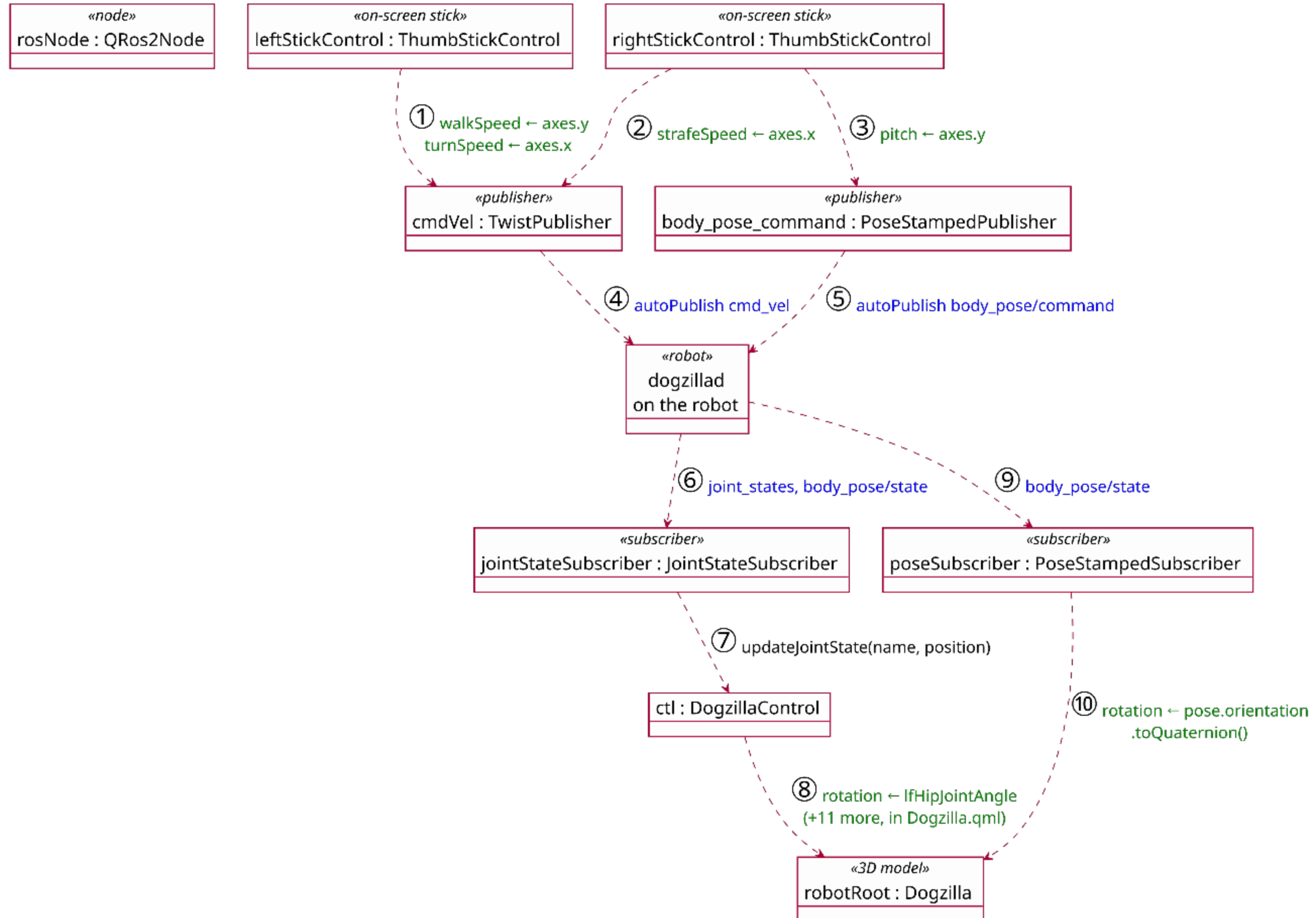
Ros2.Node {
    id: root
    nodeName: "dogzillad" // FQN /dogzilla/dogzillad
    nodeNamespace: "/dogzilla"

    TwistPublisher {
        id: twp
        topic: `${root.nodeNamespace}/vel`
        linear.x: controller.walkSpeed
        linear.y: controller.sideStepSpeed
        angular.z: controller.steerAngle
    }

    JointStatePublisher {
        id: jsp
        topic: `${root.nodeNamespace}/joint_states`
        name: [
            "lf_lower_leg_joint"

```

Motion: the digital twin's side



```
// Generated by urdf2quick3dexporter.py from 'dogzilla.urdf'
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts
import QtQuick3D
import QtQuick3D.Helpers
import QtRos2.Core as Ros2
import QtRos2.GeometryMsgs as GeomMsgs
import QtRos2.SensorMsgs as SensorMsgs
import QtRos2.StdSrvs as StdSrvs
import Dogzilla
import Dogzilla.Interfaces
```

```
Item {
    id: root
    implicitWidth: 1920
    implicitHeight: 1080

    View3D {
        id: view3D
        anchors.fill: parent

        camera: sceneCamera
        environment: sceneEnvironment

        SceneEnvironment {
            id: sceneEnvironment
            clearColor: "#222"
            backgroundMode: SceneEnvironment.Color
            antialiasingMode: SceneEnvironment.MSAA
            antialiasingQuality: SceneEnvironment.High
        }
    }
}
```

What is URDF?

- **U**nified **R**obot **D**escription **F**ormat
- industry-standard XML schema
- links and joints
- visual materials
- collision volumes
- physics: mass, moment of inertia

How to get it

- robot vendor
- export from your CAD

```
<?xml version="1.0"?>
```

```
<!--
```

```
Robot: simple_arm
```

```
Generated by: LinkForge v1.3.4
```

```
https://github.com/arounamounchili/linkforge
```

```
Date: 2026-05-29 09:25:07
```

```
-->
```

```
<robot name="simple_arm">
```

```
<!-- Materials -->
```

```
<material name="blue">
```

```
<color rgba="0.2 0.4 0.8 1" />
```

```
</material>
```

```
<material name="green">
```

```
<color rgba="0.2 0.7 0.3 1" />
```

```
</material>
```

```
<material name="grey">
```

```
<color rgba="0.55 0.55 0.55 1" />
```

```
</material>
```

```
<material name="orange">
```

```
<color rgba="0.9 0.5 0.1 1" />
```

```
</material>
```

```
<!-- Links -->
```

```
<link name="base_link">
```

```
<visual>
```

```
<origin xyz="0 0 0.05" rpy="0 0 0" />
```

```
<geometry>
```

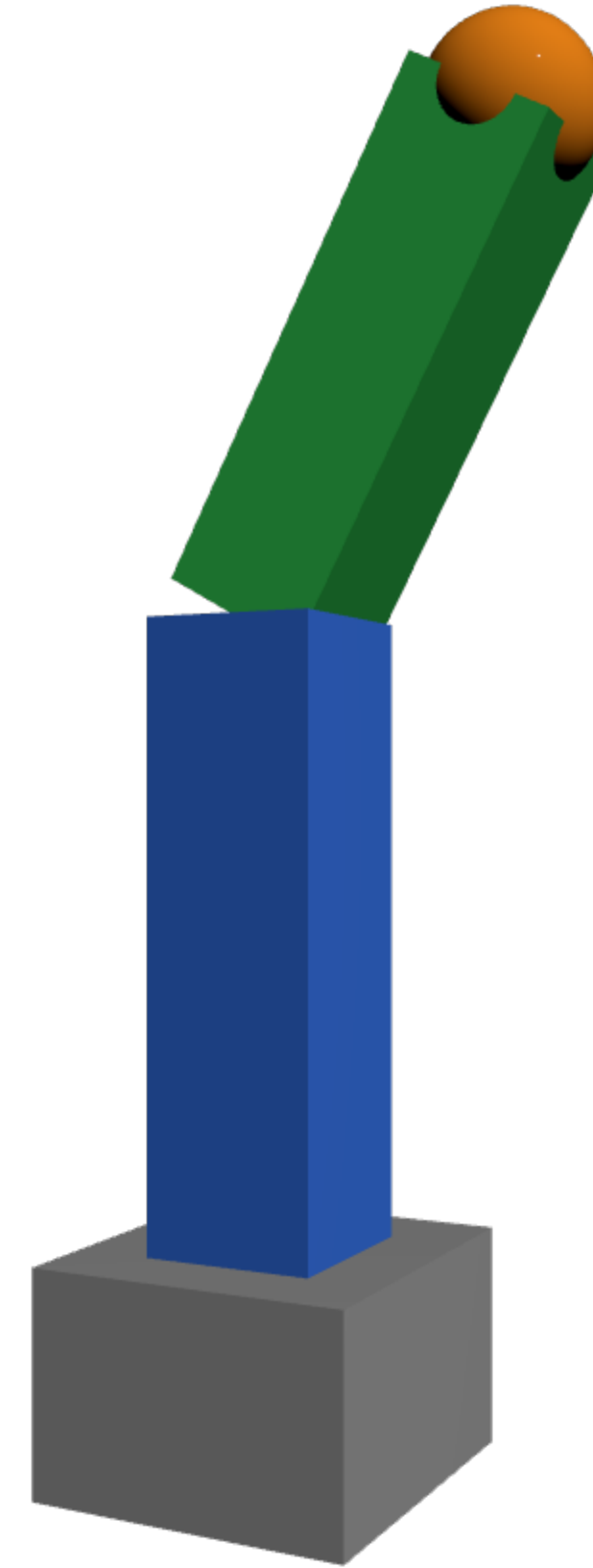
```
<box size="0.15 0.15 0.1" />
```

```
</geometry>
```

```
<material name="grey" />
```

```
</visual>
```

```
<collision>
```



```

import QtQuick
import QtQuick.Controls
import QtQuick.Controls.Universal
import QtQuick3D
import QtQuick3D.Physics
import "resources/rock" as Rock
import "resources/simplearm" as Arm

Rectangle {
    width: 280 * slideScale
    height: 640 * slideScale
    color: "#CCFFFFFF"

    View3D {
        // Generated from simple_arm.urdf: links, joints, collision volumes
        Arm.SimpleArm {
            control: Arm.SimpleArmControl {
                sendTriggerReports: true // so the rock can notice
                shoulderPanAngle: 0.5

                SequentialAnimation on shoulderLiftAngle {
                    loops: Animation.Infinite
                    NumberAnimation { from: 2; to: 2.5; duration: 1500
                                     easing.type: Easing.InOutSine }
                    NumberAnimation { from: 2.5; to: 2; duration: 1500
                                     easing.type: Easing.InOutSine }
                }
            }
        }
    }

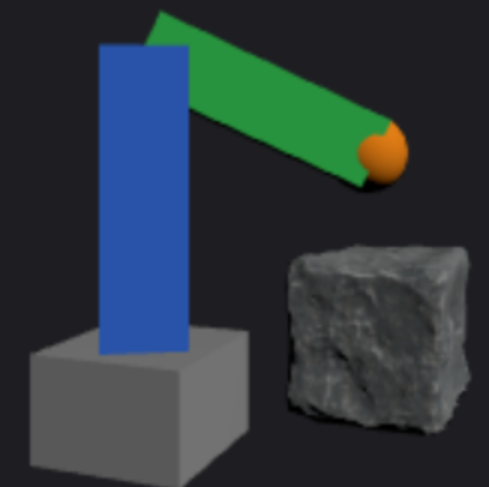
    TriggerBody {

```

Collision detection

- arm moves; rock notices collision
- rock's *visual* is a scanned mesh, *collision* volume is one cube
- URDF arm components have detailed <visual>, simplified <collision>
- TriggerBody reports overlap

0 collisions}



How to write your own Qt Quick 3D Digital Twin

```
$ urdfviewer --help
```

```
Usage: /home/rutledge/dev/qt-stabilizing-rel/qtbase/bin/urdfviewer [options] urdf or xacro destination
Import a URDF file, preview and export a Qt Quick 3D scene
```

Options:

-h, --help	Displays help on commandline options.
--help-all	Displays help, including generic Qt options.
-v, --version	Displays version information.
-b, --bridge	Preview with live ROS bridge.
-u, --upm <units>	Scene <units> per meter.
-p, --topic-prefix <prefix>	DDS topic <prefix> for robot.

Arguments:

urdf or xacro	File to open
destination	Output directory

Demo:

```
urdfviewer -b -u 1000 -p dogzilla ~/dev/qt-robotics/dogzilla/digitwin/urdf/dogzilla.urdf /tmp
```

```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts
import QtRos2.GeometryMsgs
```

```
ColumnLayout {
    id: root
    width: 200; height: 540

    Node { // in QtRos2.Core, not QQ3D
        id: rosNode
        nodeName: "simplePublisher"

        PoseStampedPublisher {
            id: posePublisher
            topic: "/dogzilla/body_pose/command"
            header.frameId: "base_link"
        }
    }
}
```

```
Label { text: `Subscribers: ${posePublisher.subscriberCount}` }
```

```
Button {
    text: rosNode.initialized ? "Publish Random Pose" : "Waiting for ROS..."
    enabled: rosNode.initialized
    onClicked: posePublisher.pose.orientation.eulerAngles = root.randomAngles()
}
```

```
TextArea {
    Layout.columnSpan: 2
    Layout.fillWidth: true
}
```

Subscribers: 0

Publish Random Pose

```
{
  "x": 0,
  "y": 0,
  "z": 0,
  "w": 1,
  "eulerAngles": {
    "x": 0,
    "y": 0,
    "z": 0
  },
  "rpy": {
    "x": 0,
    "y": 0,
    "z": 0
  },
  "rpyDegrees": {
    "x": 0,
    "y": 0,
    "z": 0
  },
  "scalar": 1
}
```



Introduction to Qt ROS - Architecture and Demo

Thanks!

shawn.rutledge@qt.io



<https://github.com/ec1oud/qt-presentations/tree/robotics>

Speech: the digital twin's side

